The types of the supplied lockable objects `LockableType1, LockableType2`,... shall conform to the `Lockable` requirements.

Effects

Acquires a lock on each of the supplied lockable objects `m1, m2`,... by an unspecified sequence of calls to the `lock(), try_lock()`, and `unlock()` members of those types that avoid deadlock.

Postconditions

The current thread owns a lock on each of the supplied lockable objects.

Throws

Any exceptions thrown by the calls to `lock(), try_lock()`, and `unlock()`.

Note

If an exception propagates out of the call to `std::lock`, then `unlock()` shall have been called for any of the objects `m1, m2`,... for which a lock has been acquired in the function by a call to `lock()` or `try_lock()`.

D.5.8. std::try_lock function template

The `std::try_lock` function template allows you to try to lock a set of lockable objects in one go, so either they are all locked or none are locked.

Declaration

```
template<typename LockableType1,typename... LockableType2>
int try_lock(LockableType1& m1,LockableType2& m2...);
```